Introduction to React Testing Library and Jest/Vitest

Section 1: Introduction

Introduction to React Testing Library and Jest/Vitest

1. React Testing Library
    1. Not just a library, also a philosophy ("opinionated")
        1. It encourages a certain set of practices
            1. The best practices
        2. Test your software the way users actually use it
            1. Not internal implementation (how software is written)
            2. We care about whether the software works the way it is supposed to
                1. How the code is written can change
                    1. The tests will pass if the software is written as per the specification.
        3. Find elements by accessibility markers, not test IDs
            1. By the way screen readers would find the elements
                1. Ensures the software is accessible
2. React Testing Library vs Jest (or Vitest)
    1. React Testing Library
        1. Provides simulated DOM for tests (no browser required)
        2. Provides ways to manipulate and examine simulated DOM
            1. Tests can do things (click buttons say), and checks to see how the DOM looks like afterwards
    2. Jest/Vitest
        1. Test runner that
            1. Finds tests
            2. Runs tests
            3. Determines whether tests pass or fail
    3. The course uses Vitest
        1. Jest: Syntax differences are shown in the course
            1. The only difference is the setup
                1. Files we use
                2. Syntax for configuration
            2. All test code written for Jest works with Vitest
            3. More advanced syntax has minor differences

Link to Course GitHub Repo

1. https://github.com/bonnie/udemy-TESTING-LIBRARY

First Test with Testing Library

1. Setup files
    1. Vite - application that can constructs a base JS frontend apps
        1. Babel/Webpack - JSX, files, imports, ...
        2. Web server with hot reload
    2. https://vitejs.dev
2. Starter from Course Repo
    1. `git clone <url>`
    2. `cd <directory>/vite-starter`
    3. `npm install`
3. This starter has:
    1. vite
    2. vitest - https://vitest.dev
        1. Test running (equivalent of Jest)
    3. React Testing Library
        1. No installation required
4. Run tests:

```
npm test
```

5. Syntax:
    1. `render()`
        1. Comes from `@testing-library/react`
        2. Constructs simulated DOM for argument component (for the JSX we pass it)
        3. Simulated DOM can be accessed via `screen` global object
    2. `screen.getByText()`
        1. Finds elements by display text
            1. `/learn react/i`
                1. Regular expression
                2. case insensitive (i)
                3. Argument could be string 'learn React'
                    1. But it must be exact string

2. `exect` - vtest syntax (same as jest)
3. Tests are automatically run in watch mode

Jest and Jest-DOM Assertions

1. Assertions
    1. `expect(linkElement).toBeInTheDocument();`
        1. `expect`
            1. Jest/Vitest global, starts the assertion
        2. `linkElement` - expect argument
            1. Assertion is asserting against
                1. Subject of the assertion
        3. `toBeInTheDocument()` - Matcher
            1. Type of assertion
            2. This matcher comes from Jest-DOM
            3. Matcher argument (optional)
                1. Refines matcher
    2. More assertion examples:
        1. `expect(element.textContent).toBe('hello');`
        2. `expect(elementsArray).toHaveLength(7);`
2. Jest-DOM
    1. It can be used with Jest or VTest
    2. Imported before each test (by test setup), makes matchers available
    3. DOM-based matchers
        1. Example: `toBeVisible()`, or `toBeChecked()`
    4. More to come!

Jest and Vitest: How Tests Work

1. React Testing Library
    1. Renders components into simulated DOM (building simulated DOM)
    2. Makes simulated DOM available for assertions and interactions
    3. Needs a test runner
        1. It finds tests, runs them, makes assertions
2. Jest vs. Vitest
    1. Why Vitest?
        1. Faster than Jest (3x - 5x)
        2. Jest isn't easy to get working with Vite
            1. Vite wants to use Vtest
    2. Jest is much easier with, say, Next.js
        1. https://github.com/vercel/next.js/tree/canary/examples/with-jest
    3. For this course, syntax is **identical** for Jest or Vitest
        1. Only setup differs
        2. Some more advanced syntax uses slightly different syntax
3. How does Jest / Vitest Work?
    1. Global `test` method has two arguments:
        1. string description
        2. test function - to run to determine whether the test is going to pass or fail
    2. Test fails if error is thrown when running function
        1. Assertions throw errors when expectation fails (failing means throwing error)
    3. No error -> tests pass
        1. Empty test passes!
4. It is the assertion that throws an error

```
expect(headingElement).not.toBeInTheDocument();
```

TDD: Test Driven Development

1. Write tests before writing code
    1. Then write code according to "spec" set by tests
2. "red-green" testing
    1. Tests fail before code is written
3. Procedure:
    1. Write "shell" function
    2. Write tests
    3. Tests fail
    4. Write code
    5. Tests pass!
4. Why TDD?
    1. Makes a huge difference in how it feels to write tests
        1. Part of the coding process, not a "chore" to do at the end
            1. Integrated into the process
                1. Doesn't feel like drudge work
    2. More efficient
        1. We test as we go along by running the app to see if it works
        2. Re-run tests "for free" after changes

1. If we make a change, we can re-run tests
    1. Free regression testing

React Testing Library Philosophy

1. What does React Testing Lirary Do?
    1. Constructs virtual DOM for testing
        1. And utilities for interacting with DOM
            1. Find elements in the DOM
            2. Click on elements in the DOM
            3. ...
    2. Allows testing without a browser
2. Types of tests
    1. Unit tests
        1. Tests one unit of code in isolation
            1. We don't test any interaction of the unit with any other units of code
    2. Integration tests
        1. Tests how multiple units work together
            1. Tests interaction between units
                1. Between Components
                2. Between Microservices
    3. Functional Tests
        1. Tests a particular function of software (feature/behavior)
            1. General behavior of software
                1. Not particular code or function
            2. Example:
                1. If we enter data into a form, and submit, we check if software does the right thing with the data
            3. Functional test as unit test:
                1. Example: If we enter invalid data, we check if entry box turns red
        2. We are not testing code, but behavior
            1. Testing how user uses the software, and not the internal implementation
        3. **React Testing library encourages functional tests**
    4. Acceptance / End-to-end (E2E) Tests
        1. Use actual browser and server (Cypress, Selenium)
            1. This is not the type of tests React Testing library is built for

Functional Testing vs Unit Testing

1. Functional Testing
    1. Different mindset from unit testing
        1. Unit Testing
            1. Isolated: mock dependencies (test versions of other dependencies are used), test internals
                1. Reason for mocking
                    1. If tests fail, it is our unit that is causing tests to fail and nothing else
                2. Testing internals
                    1. Testing differnces, the action made to the state
                        1. We don't have access to other components
                3. It is very easy to pinpoint failures
                    1. Due to isolation, we are focusing on a small portion of the code. It becomes easy to look into
            2. It is further from how users interact with software (it is less tightly coupled with the users of the software)
                1. We can have unit tests passing, but users interacting with the software would have failures
                2. We can have unit tests fail, but users interacting might not have problems
            3. They are more likely to break with refactoring
                1. Refactoring: Changing how a software is written, without changing the behavior
                    1. Since we are testing internals, the tests might fail even though the behaviour did not change
                        1. Negative aspect of Unit Tests
        2. Functional Testing
            1. Include all relevant units for a particular behavior or user flow under test
            2. Benefits
                1. Close to how users interact with the software
                    1. Passing tests means users are going to be okay with the software
                    2. Failing tests means users are going to have problems
                2. Tests are robust
                    1. If we re-factor code but behavior remains the same, tests should pass
            3. Challenges
                1. More difficult to debug failing tests
                    1. Tests are not tightly coupled to the code
                        1. It is not obvious which parts of the code are causing the tests to fail
            4. **React Testing libarary is based on the belief that the advantages of functional tests outweight the disadvantages**

TDD (Test Driven Development) vs BDD (Behavior Driven Development)

1. TDD vs BDD
    1. Quick detaur for BDD: Behavior-Driven Development
        1. Testing Library encourages testing **behavior** over **implementation**
    2. So shouldn't we be calling this BDD instead of TDD?
        1. No
            1. BDD is very explicitly defined
                1. It involves collaboration between lots of roles (developers, QA, business partners, ...)
                2. It defines a process for different groups to interact
            2. In this course, we are only developers (only one role), so we use TDD (technically)
                1. We will be testing behavior though

Testing Library and Accessibility

1. Accessibility and Finding Elements
    1. How testing library recommends finding elements on the page (opinionated) - by accessibility handles
        1. https://testing-library.com/docs/queries/about/#priority
            1. Query accessible to everyone
                1. Visual or mouse users
                2. The ones with assistive technology
            2. `getByRole` - 1st priority
                1. Role - Accessibility role
                    1. It gives a link to list of roles from MDN
            3. `getByLabelText` - for form fields
            4. `getByPlaceholderText` - Better to have label instead.
            5. `getByText` - not as good as betting by role
            6. `getByDisplayValue` - For navigating a page with filled-in values
            7. `getByAltText` - for `img`, `area`, `input`, ...
            8. `getByTitle` - Not consistently read by screen readers
            9. `getByTestId` - As a last option
                1. It is not used by users to interact with the app
    2. Fetching by role:

```
import { render, screen } from "@testing-library/react";
import App from "./App";

test("App contains correct heading", () => {
  render(<App />);
  const headingElement = screen.getByRole("heading", { name: /learn React/i }); // matches `h1` through `h6`
  expect(headingElement).toBeInTheDocument();
});
```

2. Can't fid an element like a screen reader would?
    1. Then your ap isn't friendly o screen readers

RESOURCE: Where to find code if you'd rather not write React for this course

1. https://github.com/bonnie/udemy-TESTING-LIBRARY/tree/main/react-code

Section 2: Simple App: Color Button

Overall Course Plan

1. Topics:
    1. Start with very simple React
        1. Focus on Testing Library syntax
            1. Not worry much about React
    2. First app: not much of an app
        1. Changing button color, disabling button with checkbox
    3. Introduce: Testing interactions that affect DOM, unit testing functions (a helper function)
        1. Unit testing and functional testing are not mutually exclusive
            1. Unit testing role: For individual functions which might be complicated, and we might need to test a lot of variations without involving the DOM.
    4. Eventually we build up to more complex functionality and complex tests
    5. Second app: design and order an ice-crean sundae (more realistic app)
        1. Testing more comlex user interactions (typing into text box), interactions between components
        2. Mocking server responses with Mock Service Worker
        3. Testing asyn functionality (due to service)
    6. A note about React explanations
        1. Folks come to this course at many levels
        2. Optional lectures explaining React syntax and decisions
        3. Feel free to skip!

Start Color Button App

1. Simple App
    1. Color Button
2. Requirement
    1. Button color: Red
    2. When we click, it changes to Blue
    3. When we click, it changes back to Red
3. Front-end tests build on each other (unlike backend tests)
    1. Front-end goes through flows
        1. It is okay to have multiple

4. Code: App.test.jsx

```
import { render, screen } from "@testing-library/react";
import App from "./App";
import { expect } from "vitest";

test("button starts with correct color", () => {
  render(<App />);
  const buttonElement = screen.getByRole("button", { name: /blue/i }); // If the method does not find an element, it thros an
  expect(buttonElement).toHaveClass("red");
});

test("button starts with correct text", () => {});

test("button has correct color after click", () => {});

test("button has correct color after click", () => {});
```

5. Code: App.jsx

```
import "./App.css";

function App() {
  return (
    <div>
      <button className="red">Change to blue</button>
    </div>
  );
}

export default App;
```

6. Code: App.css

```
.red {
  background-color: red;
}
```

`logRoles` method for debugging roles

1. https://testing-library.com/docs/dom-testing-library/api-debugging#logroles
    1. `logRoles` - helper function to print out a list of all implicit ARIA roles within a tree of DOM nodes
        1. Example:

```
import { logRoles } from '@testing-library/dom';

const nav = document.createElement('nav');
nav.innerHTML = `
    <ul>
        <li>Item 1</li>
        <li>Item 2</li>
        <li>Item 3</li>
    </ul>`;

logRoles(nav);
```

        2. Code:

```
import { logRoles } from "@testing-library/react";

const { container } = render(<App />);
logRoles(container);
```

Test Behavior when Clicking Button

1. `fireEvent` - **(M)** - for simple events
    1. Testing library has more involved user interaction library - User event (later)
2. Code: App.test.jsx

```
import { render, screen, fireEvent } from "@testing-library/react";
import App from "./App";
import { expect } from "vitest";

test("button click flow", () => {
  // render App
  render(<App />);

  // find the button
  const buttonElement = screen.getByRole("button", { name: /blue/i }); // If the method does not find an element, it thros an

  // check initial color
  expect(buttonElement).toHaveClass("red");

  // click the button
  fireEvent.click(buttonElement);

  // check button text
  expect(buttonElement).toHaveTextContent(/red/i);

  // check the button color
  expect(buttonElement).toHaveClass("blue");
});
```

OPTIONAL React Code: Click Button to Change Color

1. It is complicated to test for actual styles than it is to test for classes.
    1. So we want to make sure the CSS is getting interpreted as part of the tests
        1. For Vite: `vite.config.js - css: true`
        2. Jest: More work to interpret CSS
            1. Problems: Parsing CSS is slow, so if it is not required, it can be skipped
    2. Catching visual style inconsistencies is out of scope of the course
2. Code: App.test.jsx

```
import { render, screen, fireEvent } from "@testing-library/react";
import App from "./App";
import { expect } from "vitest";

test("button click flow", () => {
  // render App
  render(<App />);

  // find the button
  const buttonElement = screen.getByRole("button", { name: /blue/i }); // If the method does not find an element, it thros an

  // check initial color
  expect(buttonElement).toHaveClass("red");

  // click the button
  fireEvent.click(buttonElement);

  // check button text
  expect(buttonElement).toHaveTextContent(/red/i);

  // check the button color
  // expect(buttonElement).toHaveClass("blue"); // more straigh-forward
  // expect(buttonElement).toHaveStyle({ "background-color": "blue" }); // fails, because it gets changed to `rgb(0, 0, 255)`
  expect(buttonElement).toHaveStyle({ "background-color": "rgb(0, 0, 255)" });
});
```

Manual Acceptance Testing

1. With TDD, we don't get to see our app in action unless we run it for manual acceptance testing
2. Open the app:
    1. `npm run dev`
    2. Open `http://localhost:5173/`

Test Initial Condition of Button and Checkbox

1. A checkbox to enable or disable

2. Code: App.jsx

```
import { useState } from "react";
import "./App.css";

function App() {
  const [buttonColor, setButtonColor] = useState("red");
  const nextColor = buttonColor === "red" ? "blue" : "red";

  return (
    <div>
      <button className={buttonColor} onClick={() => setButtonColor(nextColor)}>
        Change to {nextColor}
      </button>
      <br />
      <input
        type="checkbox"
        id="disable-button-checkbox"
        defaultChecked={false}
      />
      <label htmlFor="disable-button-checkbox">Disable button</label>
    </div>
  );
}

export default App;
```

3. Code: App.test.jsx

```
test("checkbox flow", () => {
  render(<App />);

  // find elements
  const buttonElement = screen.getByRole("button", { name: /blue/i });
  const checkboxElement = screen.getByRole("checkbox", {
    name: /disable button/i,
  });

  // check initial conditions
  expect(buttonElement).toBeEnabled();
  expect(checkboxElement).not.toBeChecked();
});
```

Introduction to Code Quizzes

1. Give you spec and hints, you write tests
2. Purpose
    1. Practice what you've learned
        1. Reinforce in memory
        2. Expose areas where you have questions
            1. Ask in Q&A!
            2. Please include link to GitHub for "why is my code not working"
3. Start with lots of guidance
    1. Build to complicated code quizes with very little guidance
        1. Final exam with pretty big amount of code
4. Your choice:
    1. Code tests only (test against pre-written code provided on GitHub)
    2. Code tests and code
5. Recommendation: To code both the tests and code
    1. Dive deeper into how code and tests interact
    2. Practice debugging tests and code at the same time

Code Quiz! Confirm Button Disable on Checkbox Check

1. When checkbox is checked, button should be disabled
    1. How do you think you could change the checkbox to checked in tests?
        1. `fireEvent.click()`
            1. 2x in test: Once to disable, once to re-enable
2. Assertions on button
    1. `expect(button).toBeEnabled()`
    2. `expect(button).toBeDisabled()`
3. New test or use existing tests?
    1. Add to existing checkbox flow test

4. Guidance on React code:
    1. Checkbox controls button via boolean state:
        1. state determines value of `disabled` attribute on button
        2. Recommend calling state `disabled`, defaulting to false (more intuitive than the opposite)
    2. `onChange` for checkbox
        1. `{ (e) => setDisabled(e.target.checked) }`
5. Code:

```
test("checkbox flow", () => {
  render(<App />);

  // find elements
  const buttonElement = screen.getByRole("button", { name: /blue/i });
  const checkboxElement = screen.getByRole("checkbox", {
    name: /disable button/i,
  });

  // check initial conditions
  expect(buttonElement).toBeEnabled();
  expect(checkboxElement).not.toBeChecked();

  // click checkbox to disable button
  fireEvent.click(checkboxElement);

  // check conditions
  expect(buttonElement).toBeDisabled();
  expect(checkboxElement).toBeChecked();

  // click checkbox to enable button
  fireEvent.click(checkboxElement);

  // check conditions
  expect(buttonElement).toBeEnabled();
  expect(checkboxElement).not.toBeChecked();
});
```

6. Code: App.jsx

```
import { useState } from "react";
import "./App.css";

function App() {
  const [buttonColor, setButtonColor] = useState("red");
  const [disabled, setDisabled] = useState(false);

  const nextColor = buttonColor === "red" ? "blue" : "red";

  return (
    <div>
      <button
        className={buttonColor}
        onClick={() => setButtonColor(nextColor)}
        disabled={disabled}
      >
        Change to {nextColor}
      </button>
      <br />
      <input
        type="checkbox"
        id="disable-button-checkbox"
        defaultChecked={disabled}
        onClick={() => setDisabled((disabled) => !disabled)}
      />
      <label htmlFor="disable-button-checkbox">Disable button</label>
    </div>
  );
}

export default App;
```

Code Quiz Solution: Confirm Button Disable on Checkbox Check

1. Button bray when disabled

1. Use web color "gray"
2. Test flows (simulate possible user flows):
    1. disable button -> button is gray -> enable button -> button is red
    2. click button to change color -> disable button -> button is gray
    3. Enable button -> Button is blue
3. Assertion at the end of each flow
    1. We've written other code to assert on button color

Code Quiz! Disabled Button Turns Gray

2. Code: App.jsx

```
import { useState } from "react";
import "./App.css";

function App() {
  const [buttonColor, setButtonColor] = useState("red");
  const [disabled, setDisabled] = useState(false);

  const className = disabled ? "gray" : buttonColor;
  const nextColor = buttonColor === "red" ? "blue" : "red";

  return (
    <div>
      <button
        className={className}
        onClick={() => setButtonColor(nextColor)}
        disabled={disabled}
      >
        Change to {nextColor}
      </button>
      <br />
      <input
        type="checkbox"
        id="disable-button-checkbox"
        defaultChecked={disabled}
        onClick={() => setDisabled((disabled) => !disabled)}
      />
      <label htmlFor="disable-button-checkbox">Disable button</label>
    </div>
  );
}

export default App;
```

Unit Testing Functions

1. Function separate from components
    1. Used by several components (so it is in a central place)
    2. Complex logic
        1. Separated into separate functions
2. Unit test if
    1. Complex logic is difficult to test via functional tests
        1. Why?
            1. Too many edge cases (to realistically test with functional test cases)
                1. Overkill to fire up component and run functional test for every possible edge case
                    1. But we want to ensure the function works
3. Code: helpers.js

```
export function kebabCaseToTitleCase(colorName) {
  const colorWithSpaces = colorName.replaceAll("-", " ");
  const colorWithCaps = colorWithSpaces.replace(/\b([a-z])/g, (match) =>
    match.toUpperCase()
  );

  return colorWithCaps;
}
```

4. Code: App.test.jsx

```
describe("kebabCaseToTitleCase", () => {
  // Used to group multiple tests
  test("Works for no hiphens", () => {
    expect(kebabCaseToTitleCase("red")).toBe("Red");
  });

  test("Works for one hiphen", () => {
    expect(kebabCaseToTitleCase("midnight-blue")).toBe("Midnight Blue");
  });

  test("Works for multiple hiphens", () => {
    expect(kebabCaseToTitleCase("medium-violet-red")).toBe("Medium Violet Red");
  });
});
```

Code Quiz! Update Tests for New Color Names

1. Color starts with `MediumViolteRed` and changes to `MidnightBlue`
    1. Need to update existing tests (because spec changed)
        1. Expected: behavior changed!
2. Tests for checkbox disabling should still pass
    1. Free regression testing!
        1. All the other tests must pass (no additional effort required)
        2. **Writing tests resilient to changes is a good thing**
            1. If the spec doesn't change appreciably, tests will pass
3. Code: App.test.jsx

```
// Tests an entire flow
test("button click flow", () => {
  // render App
  render(<App />);

  // find the button
  const buttonElement = screen.getByRole("button", { name: /blue/i }); // If the method does not find an element, it thros an

  // check initial color
  expect(buttonElement).toHaveClass("medium-violet-red");

  // click the button
  fireEvent.click(buttonElement);

  // check button text
  expect(buttonElement).toHaveTextContent(/red/i);

  // check the button color
  // expect(buttonElement).toHaveClass("blue"); // more straigh-forward
  // expect(buttonElement).toHaveStyle({ "background-color": "blue" }); // fails, because it gets changed to `rgb(25, 25, 112
  expect(buttonElement).toHaveStyle({ "background-color": "rgb(25, 25, 112)" });
});

test("checkbox flow", () => {
  render(<App />);

  // find elements
  const buttonElement = screen.getByRole("button", { name: /blue/i });
  const checkboxElement = screen.getByRole("checkbox", {
    name: /disable button/i,
  });

  // check initial conditions
  expect(buttonElement).toBeEnabled();
  expect(buttonElement).toHaveClass("medium-violet-red");
  expect(checkboxElement).not.toBeChecked();

  // click checkbox to disable button
  fireEvent.click(checkboxElement);

  // check conditions
  expect(buttonElement).toBeDisabled();
  expect(buttonElement).toHaveClass("gray");
  expect(checkboxElement).toBeChecked();

  // click checkbox to enable button
  fireEvent.click(checkboxElement);

  // check conditions
  expect(buttonElement).toBeEnabled();
  expect(buttonElement).toHaveClass("medium-violet-red");
  expect(checkboxElement).not.toBeChecked();
});

describe("kebabCaseToTitleCase", () => {
  // Used to group multiple tests
  test("Works for no hiphens", () => {
    expect(kebabCaseToTitleCase("red")).toBe("Red");
  });

  test("Works for one hiphen", () => {
    expect(kebabCaseToTitleCase("midnight-blue")).toBe("Midnight Blue");
  });

  test("Works for multiple hiphens", () => {
    expect(kebabCaseToTitleCase("medium-violet-red")).toBe("Medium Violet Red");
  });
});
```

When to Unit Test

1. When to unit test?
    1. `kebabCaseToTitleCase` is pretty simple
    2. Could be covered by functional tests on button

2. For more complicated functions, unit tests help with:
    1. covering all possible edge cases
    2. To determining what caused functional tests to fail
3. This is issue with functional tests:
    1. High-level makes them resistant to refactors
        1. When we change implementation and not behavior
    2. Problems: High-level makes them difficult to diagnose
        1. When a test fails, it covers a broad section of functionality
            1. If we have a complicated function
                1. If we unit test the function, and if unit tests fail, we know that this function is causing the functional tests to fail.
                    1. Unit testing is optional

Review: Simple App

1. Test interactivity using `fireEvent` (object we import from testing-libary/react. It has methods)
2. jest-dom assertions:
    1. `toBeEnabled()`
    2. `toBeDisabled()`
    3. `toBeChecked()`
3. `getByRole` option `{ name: }`
4. Jest/Vitest `describe` to group tests
5. Unit testing functions
    1. It is appropriate if the function is complicated enough
        1. We don't want to re-render a component for every edge case we want to test

Section 3: ESLint with Testing Library, plus Prettier

ESLint and Prettier

1. ESLint and Prettier
    1. ESLint - Linter
    2. Prettier - Formatter
2. Tools that testing library provides for ESLint
    1. ESLint
        1. Popular linter for JavaScript
            1. Linter: analyzes static text and marks syntax that breaks rules
            2. Static: Analyze code as written, not what happens when code is run
3. Linking keeps code style consistent
    1. Especially for multi-engineer projects
4. Also catches errors in code
    1. Jsing variable before defining
    2. Importing from nonexistent file
    3. etc.
5. Linting vs. Formatting
    1. Formatters (like prettier) automatically format code (indents, spacing)
        1. Example: spaces around curly braces

```
import { useEffect } from 'react';
```

    2. Linters address format and style
        1. Example: Preferred assertion method
            1. `expect(checkbox).toHaveAttribute(checked);` // Against JS Dom style
                1. `expect(checkbox).toBeChecked();`
6. ESLint Plugins
    1. Plug-ins extend ESLint
    2. Testing Library and jest-dom have ESLint plugins
        1. They enforce best practices
            1. Plugin rule lets us know if we are doing something against the best practices
    3. https://github.com/testing-library/eslint-plugin-testing-library
    4. https://github.com/testing-library/eslint-plugin-jest-dom
        1. To know what the rules are & the rules that default to on or off
    5. Vitest ESLint plugin
        1. Enforces best practices
            1. Makes sure test "experiments" don't make it into CI
        2. Prevents ESLint from flagging Vitest globals like `test` and `describe` don't exist (if we don't include vitest in ESLint config)
    6. https://www.npmjs.com/package/eslint-plugin-test

ESLint for Testing Library, Jest-DOM and Vitest

1. sundae-starter
    1. It is already setup with ESLint for Test Library, Jest-DOM, and Vitest
        1. Run: `npm install`
2. vite-starter
    1. Partially set u with ESLint for Vitest
3. To start using `vite-starter`:
    1. sundae-starter README
        1. How to install and set up ESLint for Testing Library, Jest-DOM, and Vitest

Configuring ESLint and Prettier in VSCode

1. sundae-starter README
    1. Install ESLint and Prettier plugins for VSCode
    2. Configure VSCode to update your code with ESlint and Prettier suggestions on file save
    3. Link to a troubleshooting guide if you're having issues

Section 4: Sundaes On Demand: Form Review and Popover

Introduction to Sundaes on Demand

1. https://github.com/bonnie/udemy-TESTING-LIBRARY.git
2. App Summary:
    1. Choose ice cream flavors and toppings and submit order
    2. Flavors and toppings come from server
    3. Order is sent to server
    4. Server sends order number to display to the client
3. Backdro to Test...
    1. More complex user interactions
        1. Multiple form entry, moving through order phases
    2. mouseover popup (for terms and conditions)
        1. test that element disappears from DOM
    3. Simulating server response
        1. We do not need the server running to run functional tests
            1. Mock service worker
                1. Sends a mock response from server (intercepts requests to server and sends mock response)
4. Async app updates
    1. awaiting DOM changes
5. Global state via context
    1. Spoiler Alert
        1. We will not be testing context implementation
            1. Only interested in testing behavior as seen by user!
        2. Tests no different if we used Redux, Mobx, etc
        3. Only difference in the test setup
            1. Make sure component is wrapped in context
                1. This ensures functionality
                    1. Avoid errors - if it tries to access context, and it cannot find it
6. Order Entry Mock-up
    1. Order Entry
        1. Scopps options
        2. Toppings options
        3. Number of scoops
        4. Get a topping or not
        5. Order sundae
    2. Order Summary
        1. Scoops
        2. Toppings
        3. Total
        4. Confirm order
    3. Order confirmation page

Order Phase State (App-Level)

1. Keeping track of what order phase we are in
    1. inProgress - Order entry page will show
    2. review - Order summary page will show
    3. completed - Confirmation page
    4. inProgress - When Create new order button is clicked
2. Server
    1. Download from course repo
        1. https://github.com/bonnie/udemy-TESTING-LIBRARY/tree/master/sundae-server
        2. Follow instructions in README.md to install
    2. It is a RESTful API, runs on port 3030 (Node/Express)
    3. For flavors/toppings, just sends static info
        1. In a real app, would come from db
    4. For order, simply generates random order number
    5. Server not needed for functional react app testing!
        1. use mock-service-worker to mock responses from server
            1. Server is only used for spec, and for manual acceptance testing

WARNING: VSCode may look different!

1. Ignore the differences.

Code Organization and Introduction to SummaryForm

1. SummaryForm
    1. Form review and disappearance test
        1. We accept terms and conditions, and place order
2. SummaryForm Component
    1. Test and code checkbox / button (enables button)

        1. Sound familar
    2. Test and code Terms & conditions "popover"
        1. Syntax to test that element is no longer on page
    3. Later:
        1. Test and code summary text
        2. Test and code button functionality
3. Code organization
    1. Organize components by pages
        1. `tests` directory for each page
        2. Vitest will find and run an files that end in `.test.js`
    2. `/src/pages/summary`
        1. `OrderSummary.jsx`
        2. `SummaryForm.jsx`
    3. `/src/pages/summary/tests`
        1. `SummaryForm.test.jsx` (unit test file)

Code Quiz: Checkbox Enables Button

1. Wrhite tests to ensure that
    1. Checkbox is unchecked by default
    2. Checking checkbox enables button
    3. Unchecking checkbox again disables button
2. A chance to set up your own test file from scratch
    1. Use tests from last section as a model
    2. Render the `<SummaryForm />` component
3. Find checkbox and button using `{ name }` options
    1. Use mockup for "name" option value
4. Check that tests fail! Red part of red-green testing

Try Without Copy/Paste

1. Use what you remember and/or reference materials:
    1. https://testing-library.com/docs/react-testing-library/cheatsheet
    2. https://github.com/testing-library/jest-dom
2. Even if you use last section as a reference directly
    1. Learn better if you type it out than copy/paste

No need to code component to pass

1. No need to code up component to pass tests
    1. Initially we can code a `SummaryForm` component to just return a `<div>` (something to render in the tests)
2. For the actual component, we will be using react-bootstrap next lecture
    1. We are not expected to know react-bootstrap
        1. We can use the pre-written code:
            1. https://github.com/bonnie/udemy-TESTING-LIBRARY/tree/main/react-code/4-02-QUIZ-checkbox-enables-button
3. Can use other styling method (or none!) to code if you wish

    1. Order Summary Components

```
I agree _Terms and Conditions_
[Confirm order]
```

OPTIONAL React Code: SummaryForm Checkbox and Button

React Bootstrap Popover and Testing Library userEvent

1. User event is better than fireEvent
    1. Convenient APIs
        1. `hover()`
        2. `unhover()`
    2. `fireEvent` dispatches DOM events, but `user-event` simulates full interactions (user events)
        1. Which may fire multiple events and do additional checks along the way
            1. `setup()` - generates `user` object
                1. `user` object contains `hover`, `unhover` (user event simulation methods)
            2. They always return a `Promise`
                1. Needs `await`

Replace `fireEvent` with `userEvent`

1. Code: SummaryForm.test.jsx

```
import Form from 'react-bootstrap/Form';
import Button from 'react-bootstrap/Button';

export default function SummaryForm() {
    const [tcChecked, setTcChecked] = useState(false);

    const checkboxLabel = (
        <span>
            I agree to <span style={{ color: 'blue' }}> Terms and Conditions</span>
        </span>
    );

    return (
        <Form>
            <Form.Group controlId="terms-and-conditions">
                <Form.check
                    type="checkbox"
                    checked={tcChecked}
                    onChange((e) => setTcChecked(e.target.checked)}
                    label={checkboxLabel}
                />
            </Form.Group>
            <Button variant="primary" type="submit" disabled={!tcChecked}>
                Confirm order
            </Button>
        </Form>
    );
}
```

Screen Query Methods

1. `command[All]ByQueryType`
    1. `command`
        1. `get` : we expect element to be in DOM
        2. `query` : we expect element not to be in DOM
        3. `find` : we expect element to appear async
            1. When there are async updates to the DOM, and we want to wait until the update happens
    2. `All`
        1. (exclude) we expect only one match
        2. (include) we expect more than one match
            1. It gives an array of all the matches
    3. `QueryType`
        1. `Role` (most preferred - ensures accessibility)
        2. `AltText` (images)
        3. `Text` (display elements)
            1. For non-interactive display elements that don't have a special role
        4. Form elements
            1. `PlaceholderText`
            2. `LabelText`
            3. `DisplayValue`
2. Screen query references
    1. https://testing-library.com/docs/dom-testing-library/api-queries
    2. https://testing-library.com/docs/react-testing-library/cheatsheet
    3. https://testing-library.com/docs/guide-which-query
3. Which query should I use?
    1. Queries accessible to everyone
    2. Semantic queries (less consisten between browsers and assistive tech) (tests may not interact with the software the same way users would)
        1. `getByAltText`
        2. `getByTitle`
    3. Test IDs
        1. `getByTestId` : User is not going to interact with test ids

Testing Element is Not on Page: Popover Tests

1. Code: SummaryForm.test.jsx

```
test("popover response to hover", async () => {
  const user = userEvent.setup();
  render(<SummaryForm />);

  // popover starts out hidden
  const nullPopover = screen.queryByText(
    /no ice cream will actually be delivered/i
  ); // popover doesn't have a role
  expect(nullPopover).not.toBeInTheDocument();

  // popover appears on mouseover of checkbox label
  const termsAndConditions = screen.getByText(/terms and conditions/i);
  await userEvent.hover(termsAndConditions);
  const popover = screen.getByText(/no ice cream will actually be delivered/i);
  expect(popover).toBeInTheDocument();

  // popover disappears when we mouse out
  await user.unhover(termsAndConditions);
  expect(popover).not.toBeInTheDocument();
});
```

OPTIONAL React Code: Popover

1. Code: SummaryForm.jsx

```
import { useState } from "react";
import Form from "react-bootstrap/Form";
import Button from "react-bootstrap/Button";
import OverlayTrigger from "react-bootstrap/OverlayTrigger";
import Popover from "react-bootstrap/Popover";

function SummaryForm() {
  const [tcChecked, setTcChecked] = useState(false);

  const popover = (
    <Popover id="popover-basic">
      <Popover.Header as="h3">Popover right</Popover.Header>
      <Popover.Body>No ice cream will actually be delivered</Popover.Body>
    </Popover>
  );

  const checkboxLabel = (
    <span>
      I agree to
      <OverlayTrigger placement="right" overlay={popover}>
        <span style={{ color: "blue" }}>Terms and Conditions</span>
      </OverlayTrigger>
    </span>
  );

  return (
    <Form>
      <Form.Group controlId="terms-and-conditions">
        <Form.Check
          type="checkbox"
          checked={tcChecked}
          onChange={(e) => setTcChecked(e.target.checked)}
          label={checkboxLabel}
        />
      </Form.Group>
      <Button variant="primary" type="submit" disabled={!tcChecked}>
        Confirm order
      </Button>
    </Form>
  );
}

export default SummaryForm;
```

Review: Summary Form

1. (review) testing flow where checkbox controls whether button is disabled
2. mouseover for terms and conditions: `userEvent`

    1. Requires setup step: `user = userEvent.setup();`
    2. All methods are promises and must be `await` ed
    3. methods `user.hover` and `user.unhover`
3. For element starting out not on page
    1. `queryByText`
    2. `expect().not.toBeInTheDocument()`
4. So much for a tiny component!
    1. There's a lot to learn!
        1. That's why you're taking this course
        2. That last slide had a whole lot of stuff we covered
    2. Also, starting with smaller tests
        1. As tests cover larger functionality, will progress more quickly

Section 5: Simulating Server Response with Mock Service Worker

OrderEntry Server Data Introduction

Simulating Data from Server

1. Order Entry Components
    1. Scoop options come from the server
    2. Toppings come from the server
2. Scoops: `Options` sub-component
3. Toppings: `Options` sub-component

Tests

1. We test that option images render (no interactive tests)
2. Mock Service Worker
3. Mock server response for `/scoops` and `/toppings`
    1. Write 'scoops' code together
    2. 'toppings' code as code quiz

Introduction to Mock Service Worker and Handlers

Mock Service Worker

1. Purpose
    1. Intercept network calls
    2. Return specified responses
2. Prevents network calls during tests
3. Set up test conditions based on server response
    1. Test conditions depend on what server returns as options
        1. If server had responded with the response that we are telling the mock server to respond with, what would we expect the page to look like

Mock Service Worker Setup

1. `npm install msw` - not required for this project
2. Construct handlers
    1. Functions that determine what is returned for a particular URL and route
3. Construct a test server (to handle requests)
4. Make sure test server listens during all tests
    1. In setup tests file
    2. Reset after each test
5. https://mswjs.io/docs/basics/mocking-responses

Mock Service Worker Handler

```
    http.get(
        'http://localhost:3030/scoops',
        () => { return HttpResponse.json({ user: { id: 1, name: 'test' } }
    );

1. Handler Type: htp or graphql (different one)
    1. HTTP method to mock: `get`, `post`, etc.
        1. Full URL to mock - URL that goes as network request to network
            1. Response resolver function
                1. returns response object
                2. can take argmetns to get a `request` and `params`
                    1. https://mswjs.io/docs/basics/mocking-responses
                        1. `.json`
```

1. It can save a subset of the data (we can maintain the shape of the data)

Why not mock axios using Jest/Vitest?

1. Why not use Jest mocks or Sinon for the `axios` methods?
    1. Is it necessary to use Mock Service Worker?
        1. Reasons:
            1. We make two requests at the same time (scoops, and toppings on options page)
                1. It is hard to manage that with mock return values (?). (They are brittle too)
                    1. If we re-order, tests would fail, because return values were in the wrong order on the mock
                2. MSW allows returning values based on request type (POST) through handlers
                3. MSW can handle sophisticated aspects of requests like cookies, binary responses (like images)
                4. The methods can be switched with MSW (fetch to axios, say), and MSW works regardless
            2. tl;dr: (too long; didn't read)
                1. For simple apps, mocking could work, but MSW provides lots of tools for more complicated apps

Setting up the Mock Service Worker Server

1. Integration:
    1. mswjs.io/docs/integrations/node - What runs when we run jest or vtest
        1. Setup:

```
// src/mocks/node.js
import { setupServer } from 'msw/node'
import { handlers } from './handlers'

export const server = setupServer(...handlers)
```

            1. vitest setup: setupTests.js

```
import "@testing-library/jest-dom";

// vitest.setup.js
import { beforeAll, afterEach, afterAll } from "vitest";
import { server } from "./mocks/server";

// Establishing API mocking before all tests
beforeAll(() => server.listen());

// Resets any request handlers that we may add during the tests,
// so they don't affect other tests
afterEach(() => server.resetHandlers());

// Clean up after the tests are finished
afterAll(() => server.close());
```

Tests with Mock Service Worker: Scoop Options

1. When `Options` component sends a request to `/scoops` endpoint on the server, it takes scoops options, and displays them in `ScoopsOptions` component

Mock Service Worker in Tests

1. Options Component
    1. Will make a get request to server (`GET /scoop`)
        1. But, MSW will intercept the request, and sends back handler response
2. Code: Options.text.jsx

```
import { render, screen } from "@testing-library/react";
import Options from "../Options";

test("displays image for each scoop option from server", () => {
  render(<Options optionType="scoops" />);

  // find images
  const scoopImages = screen.getAllByRole("img", { name: /scoop$/i }); // alt-text
  expect(scoopImages).toHaveLength(2);

  // confirm alt text of images
  const altText = scoopImages.map((element) => element.alt);
  expect(altText).toEqual(["Chocolate scoop", "Vanilla scoop"]);
});
```

TROUBLESHOOTING: "ReferenceError: Response is not defined"

1. Node >= v18

OPTIONAL: React Code: Options and ScoopOption Components

1. We are using `axios`. Pre-installed in the project (needs to be installed in other projects)
2. Code: Options.jsx

```
import axios from "axios";
import { useEffect, useState } from "react";
import Row from "react-bootstrap/Row";
import ScoopOption from "./ScoopOption";

export default function Options({ optionType }) {
  const [items, setItems] = useState([]);

  // optionType is 'scoops' or 'toppings'
  useEffect(() => {
    axios
      .get(`http://localhost:3030/${optionType}`)
      .then((response) => setItems(response.data))
      .catch((error) => {
        // TODO: handle error response
      });
  }, [optionType]);

  // TODO: replace `null` with ToppingOption when available
  const ItemComponent = optionType === "scoops" ? ScoopOption : null;

  const optionItems = items.map((item) => (
    <ItemComponent
      key={item.name}
      name={item.name}
      imagePath={item.imagePath}
    />
  ));

  return <Row>{optionItems}</Row>;
}
```

3. Code: ScoopOption.jsx

```
import Col from "react-bootstrap/Col";

export default function ScoopOption({ name, imagePath }) {
  return (
    <Col xs={12} sm={6} md={4} lg={3} style={{ textAlign: "center" }}>
      <img
        style={{ width: "75%" }}
        src={`http://localhost:3030/${imagePath}`}
        alt={`${name} scoop`}
      />
    </Col>
  );
}
```

4. Code: Options.text.jsx

```
import { render, screen } from "@testing-library/react";
import Options from "../Options";

test("displays image for each scoop option from server", () => {
  render(<Options optionType="scoops" />);

  // find images
  const scoopImages = screen.getAllByRole("img", { name: /scoop$/i }); // alt-text
  expect(scoopImages).toHaveLength(2);

  // confirm alt text of images
  const altText = scoopImages.map((element) => element.alt);
  expect(altText).toEqual(["Chocolate scoop", "Vanilla scoop"]);
});
```

Using `await findBy` to Find Elements that Populate Asynchronously

1. The error is because of asynchronicity
    1. We are populating response in async way
        1. Solution: `await`, `findBy`
            1. When we are waiting for something to appear asynchronously on the page, we must use `await`, and `findBy`
2. Code: Options.test.jsx

```
import { render, screen } from "@testing-library/react";
import Options from "../Options";

test("displays image for each scoop option from server", async () => {
  render(<Options optionType="scoops" />);

  // find images
  const scoopImages = await screen.findAllByRole("img", { name: /scoop$/i }); // alt-text
  expect(scoopImages).toHaveLength(2);

  // confirm alt text of images
  const altText = scoopImages.map((element) => element.alt);
  expect(altText).toEqual(["Chocolate scoop", "Vanilla scoop"]);
});
```

Code Quiz! Topping Options from Server

1. Review of "scoops" Testing
    1. Mock Service Worker mimics response from server
        1. Construct handler
        2. Construct server (uses the handler to send the response)
        3. update `setupTests` to listen for requests
            1. Before all tests
            2. Reset handlers if they have gotten changed
2. `getAllByRole`
    1. Search for more than one match to role
        1. Axios call returned asynchronously
            1. `await findAllByRole`
                1. For asynchronous DOM update of elements

Use scoops as a model

1. Type out rather than copy/paste for better learning
2. Add handler for `/toppings` route
    1. Handler cna return three toppings:

```
[
    { name: 'Cherries', imagePath: '/images/cherries.png' },
    { name: 'M&Ms', imagePath: '/images/m-and-ms.png' },
    { name: 'Hot fudge', imagePath: '/images/hot-fudge.png' },
]
```

3. Write tests in Options.test.jsx
4. `name` option can be `/topping$/i`
5. Update `Options.jsx` and construct `ToppingOption.jsx`

Error Server Response Planning

1. Alert banner component

Server Errors: Planning

1. Fill in the `catch` statement we left as TODO
2. Display AlertBanner component if `axios` call throws error
    1. Instead of content from server
3. Use simple react-bootstrap alert
    1. https://react-bootstrap.netlify.app/components/alerts
        1. role: `alert`
4. By default (server handlers return non-error response)
    1. We need to override error response for particular tests

Jest/Vitest Debugging Tools

1. Jest/Vitest debugging tools in this section
    1. Running only one test file

    2. Running only one test within a file
        1. To isolate during debugging
2. To give us something to debug:
    1. Going to write code I know won't pass

Simulating Server Error Response in Tests

1. `AlertBanner` s
2. Code: OrderEntry.test.jsx

```
import { http, HttpResponse } from "msw";
import { server } from "../../../mocks/server";

import { render, screen } from "@testing-library/react";
import OrderEntry from "../OrderEntry";

test("handles errors for scoops and toppings routes", async () => {
  server.resetHandlers(
    http.get("http://localhost:3030/scoops", () => {
      return new HttpResponse(null, { status: 500 });
    }),
    http.get("http://localhost:3030/toppings", () => {
      return new HttpResponse(null, { status: 500 });
    })
  );

  render(<OrderEntry />);

  const alerts = await screen.findAllByRole("alert", {
    name: "An unexpected error occurred. Please try again later.",
  });

  expect(alerts).toHaveLength(2);
});
```

OPTIONAL React Code: Alert Banner for Options Server Error

1. Code: OrderEntry.jsx

```
import Options from "./Options";

function OrderEntry() {
  return (
    <div>
      <Options optionType="scoops" />
      <Options optionType="toppings" />
    </div>
  );
}

export default OrderEntry;
```

2. Code: OrderEntry.test.jsx

```
import { http, HttpResponse } from "msw";
import { server } from "../../../mocks/server";

import { render, screen } from "@testing-library/react";
import OrderEntry from "../OrderEntry";

test("handles errors for scoops and toppings routes", async () => {
  server.resetHandlers(
    http.get("http://localhost:3030/scoops", () => {
      return new HttpResponse(null, { status: 500 });
    }),
    http.get("http://localhost:3030/toppings", () => {
      return new HttpResponse(null, { status: 500 });
    })
  );

  render(<OrderEntry />);

  const alerts = await screen.findAllByRole("alert", {
    name: "An unexpected error occurred. Please try again later.",
  });

  expect(alerts).toHaveLength(2);
});
```

Tools for Debugging Tests

1. Restricting the number of tests running
    1. If `Options.test.jsx` is saved, only that file is run
    2. If `setupTests.js` is saved, all the tests are run (all the tests are affected by it)
        1. Hit `h`
        2. Hit `p` - to filter by a filename
            1. File name pattern: OrderEntry
    3. Running only one test:

    ```
    test.only(...)
    ```

    4. Skipping one test:

    ```
    test.skip(...)
    ```

    5. Using `logRoles`

    ```
    const container = render("<App />");
    // ...
    logRoles(container);
    ```

    6. Other debugging techniques: https://mswjs.io/docs/runbook

OPTIONAL: Why doesn't "name" work with the role "alert"?

1. Bootstrap alert doesn't have `name` value
    1. It has something to do with how React Bootstrap renders alert
        1. Alert is an `HTMLDivElement` with an array of children that contains on `Text` element
            1. `Text` child element in an array is not accessible as a name for `findAllByRole`
    2. How to fix it?
        1. Omit the name option
        2. Search by text

        ```
        const alerts = await screen.findAllByText(
            // An unexpected error occurred. Please try again later
        );
        ```

Review: Server Error Response and Test Debugging Tools

1. Override Mock Service Worker response for individual tests (for error conditions)
2. Misleading Unable to find role="alert" error
3. Isolate file by typing p in Jest/Vitest watch mode

4. Isolate test within file with `text.only` or `text.skip`
5. MSW debugging
    1. "Intercepted a request without a matching request handler" error (typo in URL say)
    2. MSW docs debugging "runbook": https://mswjs.io/docs/runbook/

Section 6: Testing Components Wrapped in a Provider

Intro to Tests for Total and Subtotals

Subtotals and Totals

1. React Context
    1. Options are stored in React Context
        1. It doesn't matter to our tests (internal details doesn't matter)

Entering Text Input: Subtotal Tests

1. Before updating a text element, it is better to clear the element. We don't know what's in it before or after
2. Code: TotalUpdates.test.jsx

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import Options from "../Options";

test("update scoop subtotal when scoops change", async () => {
  const user = userEvent.setup();
  render(<Options optionType="scoops" />);

  // make sure total starts out at $0.00
  const scoopSubtotal = screen.getByText("Scoops total: $", { exact: false });
  expect(scoopSubtotal).toHaveTextContent("0.00");

  // update vanilla scoops to 1, and check subtotal
  const vanillaInput = await screen.findByRole("spinbutton", {
    name: "Vanilla",
  });

  await user.clear(vanillaInput);
  await user.type(vanillaInput, "1");
  expect(scoopSubtotal).toHaveTextContent("2.00");

  // update chocolate scoops to 2, and check subtotal
  const chocolateInput = await screen.findByRole("spinbutton", {
    name: "Chocolate",
  });

  await user.clear(chocolateInput);
  await user.type(chocolateInput, "2");
  expect(chocolateInput).toHaveTextContent("6.00");
});
```

OPTIONAL: React Code: OrderDetails Context

Context File

1. Kent C. Dodds pattern to set up context with embedded state
    1. https://kentcdodds.com/blog/application-state-management-with-react
        1. Context file
            1. make context with: `createContext`
            2. make custom hook with: `useContext`
                1. It returns context (as long as it is within a provider)
            3. make context provider with internal state via `useState` statement (that is used for creation of internal state)
                1. Provider value will expose getters and setters for the state
                    1. The context provider has state built in
                        1. Example: The role of the state
                            1. Track the no of scoops, toppings, which type user has ordered
        2. Export the custom hook (for component state import) and provider (we wrap any components that need it with the provider)
            1. Example: `OrderEntry`, `OrderSummary`
            2. Example: `OrderConfirmation` doesn't need it
2. Code: OrderDetails.jsx

```
import { createContext, useContext, useState } from "react";
import { pricePerItem } from "../constants";

const OrderDetails = createContext();

// construct custom hook to check whether we're in a provider
export function useOrderDetails() {
  const contextValue = useContext(OrderDetails);

  if (contextValue) {
    throw new Error(
      "userOrderDetails must be called from within an OrderDetailsProvider"
    );
  }

  return contextValue;
}

export function OrderDetailsProvider(props) {
  const [optionCounts, setOptionCounts] = useState({
    scoops: {}, // example: { Chocolate: 1, Vanilla: 2 }
    toppings: {}, // exmaple: { "Gummy Bears": 1 }
  });

  function updateItemCount(itemName, newItemCount, optionType) {
    // make a copy of existing state
    const newOptionCounts = { ...optionCounts };

    // update the copy with the new information
    newOptionCounts[optionType][itemName] = newItemCount;

    // update the state with the updated copy
    setOptionCounts(newOptionCounts);
  }

  function resetOrder() {
    setOptionCounts({ scoops: {}, toppings: {} });
  }

  // utility function to derive total from optionCoutns state value
  function calculateTotal(optionType) {
    // get an array of counts for the option type (for example, [1, 2])
    const countsArray = Object.values(optionCounts[optionType]);

    // total the values in the array of counts for the number of items
    const totalCount = countsArray.reduce((total, value) => total + value, 0);

    // multiply the total number of items by the price for this item type
    return totalCount * pricePerItem[optionType];
  }

  const totals = {
    scoops: calculateTotal("scoops"),
    toppings: calculateTotal("toppings"),
  };

  const value = { optionCounts, totals, updateItemCount, resetOrder };

  return <OrderDetails.Provider value={value} {...props} />;
}
```

OPTIONAL React Code: Use Context to Display Scoops Subtotal

1. OrderSummary.jsx

```
import SummaryForm from "./SummaryForm";
import { useOrderDetails } from "../../contexts/OrderDetails";
import { formatCurrency } from "../../utilities";

export default function OrderSummary() {
  const { totals, optionCounts } = useOrderDetails();

  const scoopArray = Object.entries(optionCounts.scoops); // [["Chocolate", 2], ["vanilla", 1]]
  const scoopList = scoopArray.map(([key, value]) => (
    <li key={key}>
      {value} {key}
    </li>
  ));

  const toppingArray = Object.keys(optionCounts.toppings); // ["M&Ms", "Gummy bears"]
  const toppingList = toppingArray.map((key) => <li key={key}>{key}</li>);

  return (
    <div>
      <h1>Order Summary</h1>
      <h2>Scoops: {formatCurrency(totals.scoops)}</h2>
      <ul>{scoopList}</ul>
      <h2>Toppings: {formatCurrency(totals.toppings)}</h2>
      <ul>{toppingList}</ul>
      <SummaryForm />
    </div>
  );
}
```

2. Code: SummaryForm.jsx

```
import { useState } from "react";
import Form from "react-bootstrap/Form";
import Button from "react-bootstrap/Button";
import OverlayTrigger from "react-bootstrap/OverlayTrigger";
import Popover from "react-bootstrap/Popover";

function SummaryForm() {
  const [tcChecked, setTcChecked] = useState(false);

  const popover = (
    <Popover id="popover-basic">
      <Popover.Header as="h3">Popover right</Popover.Header>
      <Popover.Body>No ice cream will actually be delivered</Popover.Body>
    </Popover>
  );

  const checkboxLabel = (
    <span>
      I agree to
      <OverlayTrigger placement="right" overlay={popover}>
        <span style={{ color: "blue" }}>Terms and Conditions</span>
      </OverlayTrigger>
    </span>
  );

  return (
    <Form>
      <Form.Group controlId="terms-and-conditions">
        <Form.Check
          type="checkbox"
          checked={tcChecked}
          onChange={(e) => setTcChecked(e.target.checked)}
          label={checkboxLabel}
        />
      </Form.Group>
      <Button variant="primary" type="submit" disabled={!tcChecked}>
        Confirm order
      </Button>
    </Form>
  );
}

export default SummaryForm;
```

Adding Context to Test Setup

1. Code: Add provider as a wrapper

```
render(<Options optionType="scoops" />, { wrapper: OrderDetailsProvider }); // accepts any provider needed for the tests
```

    1. There are other places where the wrapper is required, and we don't want to add the wrapper everywhere

Creating Custom Render to Wrap in Provider By Default

1. Applying context provider globally
    1. testing-library.co/docs/react-testing-library/setup
        1. Custom render - includes a wrapper
            1. `import { render, fireEvent } from '../test-utils';`
        2. test-utils.js
            1. We can use many providers as wrappers

Review: Scoops Subtotal with Context

1. Review of Tests
    1. `getByText` to find total
        1. `exact` option set to `false` - partial string needs to be matched
    2. Number inputs
        1. `await` and `findBy` (coming from server async)
            1. For async data
        2. `spinbutton` role
        3. `userEvent.clear` - to clear existing text
        4. `userEvent.type` - to enter number (takes a string)
    3. `wrapper` option to `render` to apply context provider
    4. Redefine Testing Library `render` to access universally

Code Quiz! Toppings Subtotal

1. Write tests for toppings subtotal
    1. Not easy! Putting a lot of concepts together here
        1. Be patient with yourself, and keep referring to the scoops example
    2. Tests can go in `src/pages/order/tests/totalUpdates.test.jsx` \
    3. Look at the mockup for inputs
        1. Instead of numbers, toppings are either checked or un-checked
2. What to test?
    1. Assert on default toppings subtotal
    2. Find and tick one box, assert on updated subtotal
        1. See `src/mocks/handlers.js` for server response (which toppings)
        2. Use `await` and `find` for checkbox (async)
    3. Tick another box on, assert on subtotal
        1. Make sure code can handle two simultaneous boxes
    4. Tick one of the boxes off (click it again) and assert on subtotal
        1. Make sure coe can handle box checked then un-checked
    5. `useEvent` reminders: `setup()` and `await` (always returns a promise)
3. Coding to pass tests
    1. Would recommend using pre-written code on this one
        1. No need to learn react-bootstrap unless you want
4. If you choose to write code to validate that tests pass:
    1. Should not have to make any changes to Options.jsx
        1. Logic for counting items and calculating subtotal in Options.jsx is alreay written
        2. Re-usable without changes for toppings
    2. Call `updateItemCount` , with ` (checkbox on) or 0 (checkbox off)
    3. Update ToppingOption.jsx to include names and checkboxes
        1. `onChange` handler
        2. Use checkbox from `OrderSummary.jsx` as a model

OPTIONAL React Code: Toppings Checkboxes

1. Code: ToppingOption.jsx

```
import Col from "react-bootstrap/Col";
import Row from "react-bootstrap/Row";
import Form from "react-bootstrap/Form";
import { useOrderDetails } from "../../contexts/OrderDetails";

function ToppingOption({ name, imagePath }) {
  const { updateItemCount } = useOrderDetails();
  const handleChange = (e) =>
    updateItemCount(name, e.target.checked ? 1 : 0, "toppings");
  return (
    <Col xs={12} sm={6} md={4} lg={3} style={{ textAlign: "center" }}>
      <img
        style={{ width: "75%" }}
        src={`http://localhost:3030/${imagePath}`}
        alt={`${name} topping`}
      />
      <Form.Group controlId={`${name}-topping-checkbox`}>
        <Form.Check type="checkbox" onChange={handleChange} label={name} />
      </Form.Group>
    </Col>
  );
}

export default ToppingOption;
```

Note on expected warning in the next lecture

1. If React Testing Library < v14

Code Quiz! Grand Total

1. Grand Total
    1. Should we do a "black box" test (not consider implementation)?
        1. For example:
            1. First updates scoops, then toppings
            2. Sould we also test updating toppings first then scoops?
            3. We know from impelmentation that it shouldn't make a difference
            4. Users should be able to do either, and we might change implementation
    2. Do test functions need to be `async` ?
        1. Yes, options still need to load from server/mock service worker
        2. Await both the scoop element and another await on the topping element
            1. They're separate operations and either could finish first
2. How to find element
    1. From mockups, grand total should be same size as titles (

)
        1. **we can search using the `heading` role**
            1. **Include the test in the `name` option**

        ---

    2. Note: `{ exact: false }` is not an option for `byRole`
        1. Either use `*byRole` and regular exression for `name` option, or
            1. `screen.getByRole('heading', { name: /grand total: \$/i});`
        2. `byText` and `{ exact: false }`
            1. Disadvantage: Cannot use the role
            2. `screen.getByText('Grand total: $', { exact: false });`
    3. Igore this error:
        1. `Warning: An update to Options inside a test was not wrapped in act(...)` .
            1. Ignore it for now
3. Coding to pass tests:
    1. Add grand total in `OrderEntry.jsx`
    2. Not a huge change
        1. Add `<h2>` with grand total from context
            1. Add `scoops` , and `toppings`
            2. Use `formatCurrency` to format the value

OPTIONAL: "Not wrapped in act()..." Warning

OPTIONAL: Why is the explicit unmount needed

Manual Acceptance Testing

Section 7: Final Exam: Order Phases

Introduction to Final Exam: Order Phases

Tips for the new MSW Handler

Debugging Tips

OPTIONAL React Hints for Order Phase Coding

NOTE: outdated code in the next lecture

OPTIONAL React Code: Order Phases

Review: Final Exam, and Introduction to Optional Practice

Common Mistakes with React Testing Library

Section 8: Optional Extra Practice

Standard Questions for New Tests and Introduction to Exercises

Conditional Toppings Section on Summary Page

Disable Order Button if No Scoops Ordered

Red Input Box for Invalid Scoop Count

No Scoops Subtotal Update for Invalid Scoop Count

Server Error on Order Confirmation Page

Congratulations and Thank You!

Section 9: Bonus

Coupons!

1. https://bonnie.dev/courses